

DOCUMENTACIÓN TÉCNICA DE LABORATORIO

Laboratorio 01: Diseño e Implementación de Objetos en SQL Server 2025

Título del laboratorio	Laboratorio 01: Diseño e Implementación de Objetos en SQL Server 2025
Entorno de ejecución	Microsoft SQL Server 2025 (RTM - 17.0.1135.8) · SQL Server Management Studio (SSMS) v20.x · Windows 11
Base de datos	EcommerceDB
Repositorio oficial	https://github.com/HugoAlcaide/master-labs
Fecha de realización	21 de septiembre de 2026

1 Creación de la Base de Datos

Inicialización del contenedor transaccional `EcommerceDB` para la plataforma de comercio electrónico.

```
CREATE DATABASE EcommerceDB;  
GO  
  
USE EcommerceDB;  
GO
```

T-SQL

Resultado de ejecución:

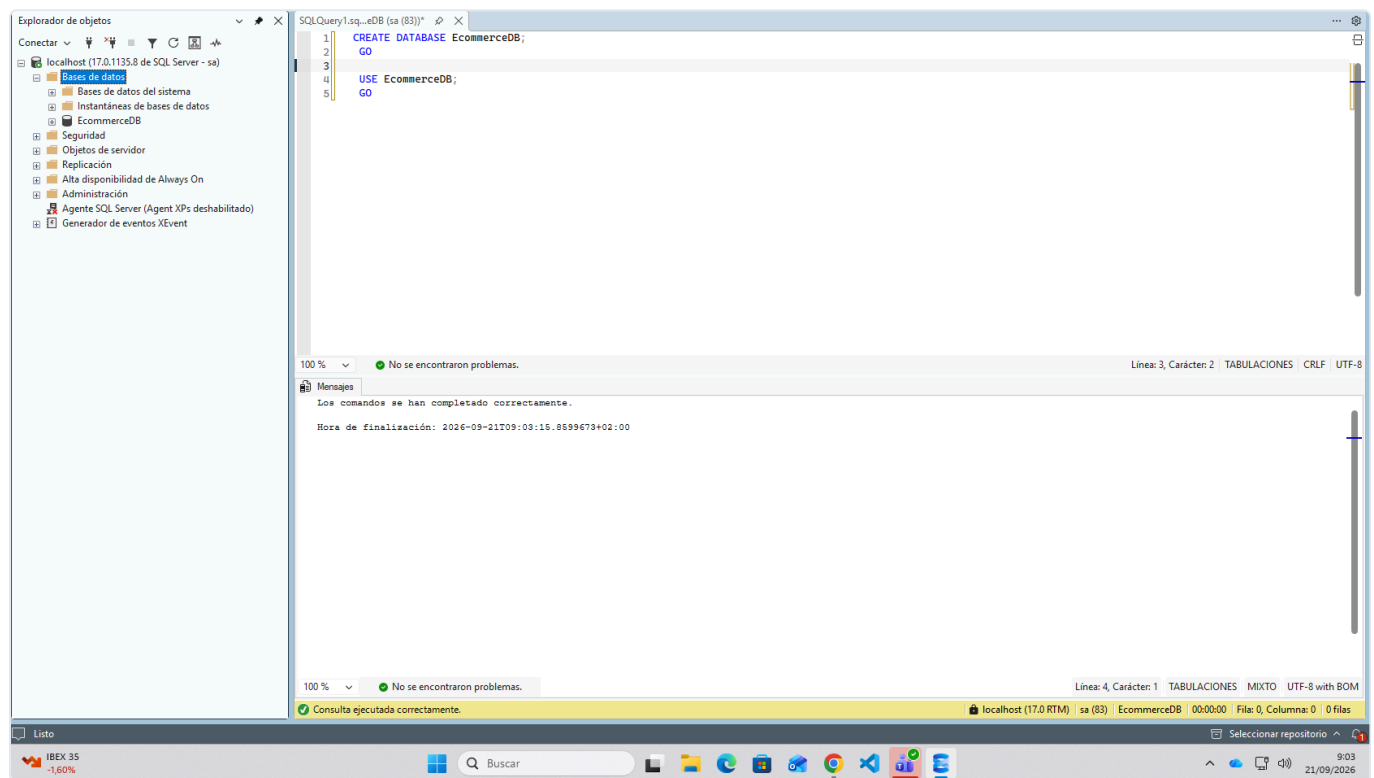


Figura 1. Evidencia en SSMS de la creación de la base de datos EcommerceDB.

2 Creación de Tablas Principales con Restricciones e Índices

Definición de las entidades maestras `Supplier`, `Category` y `Product`. Se aplican claves primarias subrogadas (`IDENTITY`), restricciones de unicidad (`UNIQUE`), valores predeterminados (`DEFAULT`), comprobaciones lógicas (`CHECK`) y claves foráneas (`FOREIGN KEY`).

T-SQL

```
USE EcommerceDB;
GO

-- Create Supplier table
CREATE TABLE Supplier (
    SupplierID INT PRIMARY KEY IDENTITY(1,1),
    SupplierName NVARCHAR(100) NOT NULL UNIQUE,
    Country NVARCHAR(50) NOT NULL,
    Email NVARCHAR(100),
    Phone NVARCHAR(20),
    CreatedDate DATETIME2 DEFAULT GETUTCDATE()
);

-- Create Category table
CREATE TABLE Category (
    CategoryID INT PRIMARY KEY IDENTITY(1,1),
    CategoryName NVARCHAR(100) NOT NULL UNIQUE,
    Description NVARCHAR(500)
);

-- Create Product table with constraints
CREATE TABLE Product (
    ProductID INT PRIMARY KEY IDENTITY(1,1),
    ProductName NVARCHAR(100) NOT NULL,
    CategoryID INT NOT NULL,
    SupplierID INT NOT NULL,
    BasePrice DECIMAL(10,2) NOT NULL,
    StockQuantity INT NOT NULL DEFAULT 0,
    CreatedDate DATETIME2 DEFAULT GETUTCDATE(),
    CHECK (BasePrice > 0),
    CHECK (StockQuantity >= 0),
    FOREIGN KEY (CategoryID) REFERENCES Category(CategoryID),
    FOREIGN KEY (SupplierID) REFERENCES Supplier(SupplierID)
);

-- Create indexes
CREATE INDEX IX_Category ON Product(CategoryID);
CREATE INDEX IX_Supplier ON Product(SupplierID);
GO
```

Resultado de ejecución:

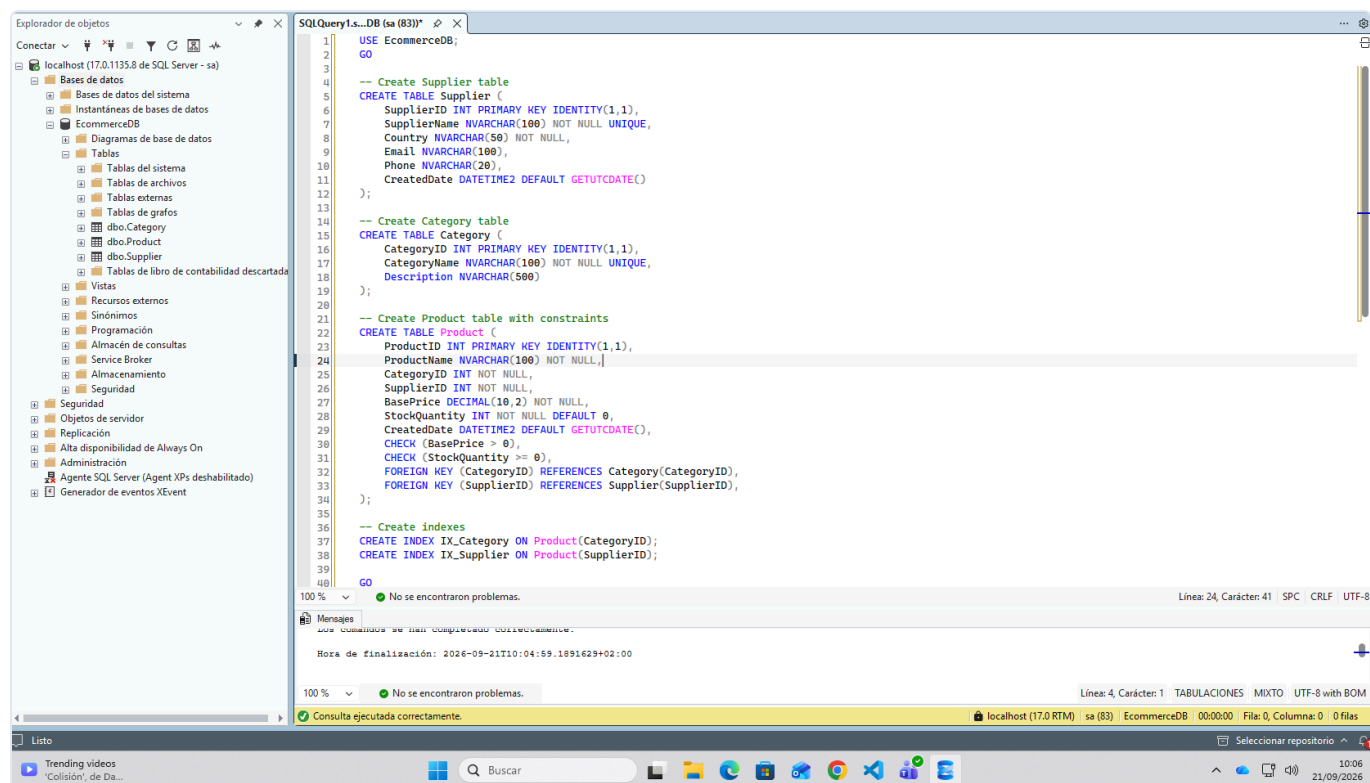


Figura 2. Evidencia en SSMS de la creación de las tablas principales con sus restricciones e índices.

3 Inserción de Datos Iniciales en Tablas Maestras

Carga de registros de prueba para validar relaciones e integridad referencial entre proveedores, categorías y productos.

```
USE EcommerceDB;
GO

-- Insert sample suppliers
INSERT INTO Supplier (SupplierName, Country, Email, Phone)
VALUES
    ('Contoso Supplies', 'USA', 'contact@contoso.com', '555-0100'),
    ('Fabrikam Inc', 'Canada', 'sales@fabrikam.com', '555-0200');

-- Insert sample categories
INSERT INTO Category (CategoryName, Description)
VALUES
    ('Electronics', 'Electronic devices and accessories'),
    ('Clothing', 'Apparel and fashion items');

-- Insert sample products
INSERT INTO Product (ProductName, CategoryID, SupplierID, BasePrice, StockQuantity)
VALUES
    ('Wireless Mouse', 1, 1, 29.99, 100),
    ('Cotton T-Shirt', 2, 2, 19.99, 250);
GO
```

T-SQL

Resultado de ejecución:

Explorador de objetos

Conectar

localhost (17.0.1135.8 de SQL Server - sa)

Bases de datos

Bases de datos del sistema

Instantáneas de bases de datos

EcommerceDB

Diagramas de base de datos

Tablas

Tablas del sistema

Tablas de archivos

Tablas externas

Tablas de grafos

dbo.Category

dbo.Product

dbo.Supplier

Tablas de libro de contabilidad descartada

Vistas

Recursos externos

Sinónimos

Programación

Almacén de consultas

Service Broker

Almacenamiento

Seguridad

Seguridad

Objetos de servidor

Replicación

Alta disponibilidad de Always On

Administración

Agente SQL Server (Agent XPs deshabilitado)

Generador de eventos XEvent

SQLQuery1.sql...eDB (sa (83))*

SQLQuery2.s...DB (sa (52))*

```
1 USE EcommerceDB;
2 GO
3
4 -- Insert sample suppliers
5 INSERT INTO Supplier (SupplierName, Country, Email, Phone)
6 VALUES
7     ('Contoso Supplies', 'USA', 'contact@contoso.com', '555-0100'),
8     ('Fabrikam Inc', 'Canada', 'sales@fabrikam.com', '555-0200');
9
10 -- Insert sample categories
11 INSERT INTO Category (CategoryName, Description)
12 VALUES
13     ('Electronics', 'Electronic devices and accessories'),
14     ('Clothing', 'Apparel and fashion items');
15
16 -- Insert sample products
17 INSERT INTO Product (ProductName, CategoryID, SupplierID, BasePrice, StockQuantity)
18 VALUES
19     ('Wireless Mouse', 1, 1, 29.99, 100),
20     ('Cotton T-Shirt', 2, 2, 19.99, 250);
21 GO
```

100 %

No se encontraron problemas.

Mensajes

(2 filas afectadas)

(2 filas afectadas)

(2 filas afectadas)

Hora de finalización: 2026-09-21T10:07:04.4269291+02:00

Figura 3. Evidencia en SSMS de la inserción de datos de prueba en las tablas maestras.

4 Implementación de Tablas Temporales (Historial de Precios)

Implementación de control de cambios temporal con control del sistema (`SYSTEM_VERSIONING = ON`) para registrar auditorías de precios de forma nativa.

T-SQL

```
USE EcommerceDB;
GO

-- Create Price History table with temporal versioning
CREATE TABLE ProductPrice (
    PriceID INT PRIMARY KEY IDENTITY(1,1),
    ProductID INT NOT NULL,
    CurrentPrice DECIMAL(10,2) NOT NULL,
    EffectiveDate DATE,
    SysStartTime DATETIME2 GENERATED ALWAYS AS ROW START HIDDEN,
    SysEndTime DATETIME2 GENERATED ALWAYS AS ROW END HIDDEN,
    PERIOD FOR SYSTEM_TIME (SysStartTime, SysEndTime),
    FOREIGN KEY (ProductID) REFERENCES Product(ProductID)
) WITH (SYSTEM_VERSIONING = ON);
GO

-- Insert initial price data
INSERT INTO ProductPrice (ProductID, CurrentPrice, EffectiveDate)
VALUES (1, 99.99, '2025-01-01'), (2, 149.99, '2025-01-01');

-- Update price (creates history entry)
UPDATE ProductPrice SET CurrentPrice = 109.99 WHERE ProductID = 1;
GO
```

Resultado de creación y actualización temporal:

The screenshot displays the SQL Server Enterprise Manager (SSMS) interface. On the left, the 'Explorador de objetos' (Object Explorer) shows the 'EcommerceDB' database structure, including tables like 'dbo.Category', 'dbo.Product', and 'dbo.Supplier'. The right pane shows the execution of the T-SQL scripts. The messages pane at the bottom indicates that the scripts were executed successfully, with messages such as '(2 filas afectadas)' (2 rows affected) and '(1 fila afectada)' (1 row affected). The final message shows the completion time: 'Hora de finalización: 2026-09-21T10:30:10.8136626+02:00'.

Figura 4. Evidencia en SSMS de la creación de la tabla temporal y de la actualización de precios.

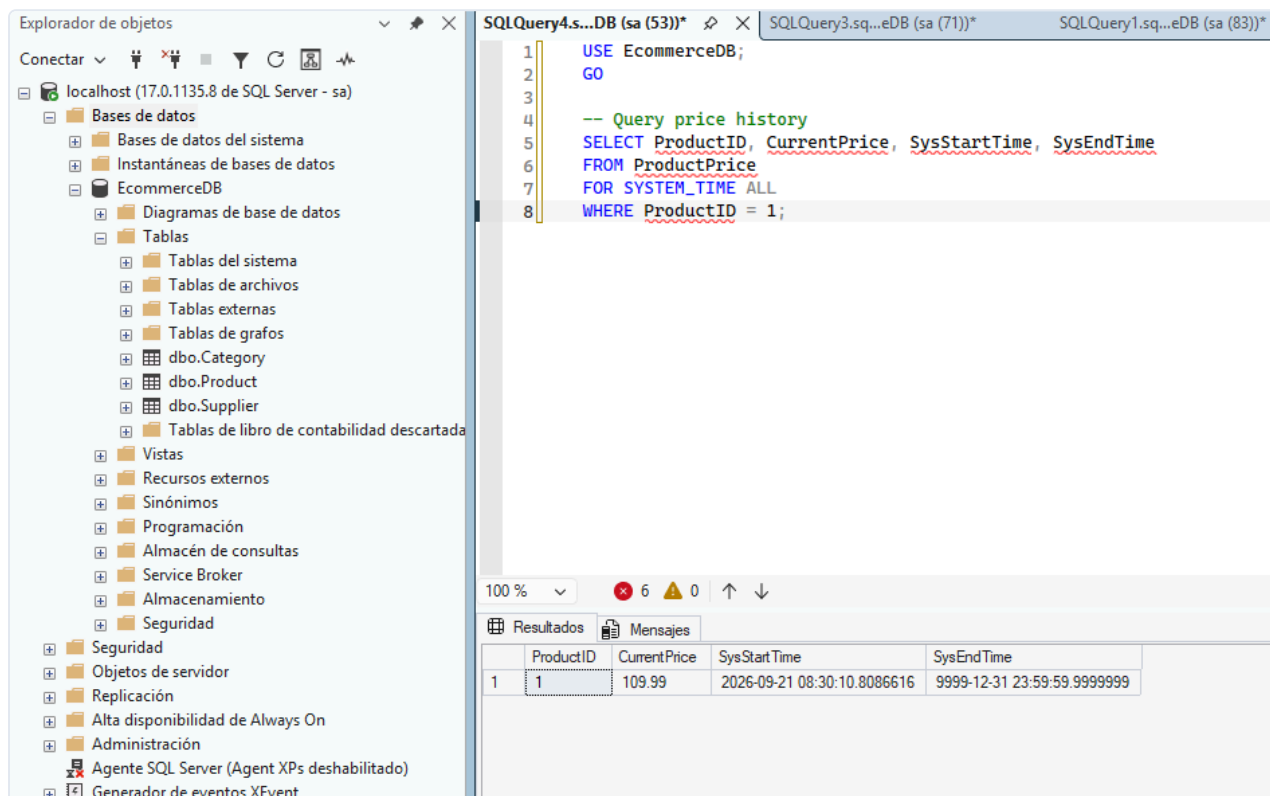
Consulta de trazabilidad histórica mediante la cláusula `FOR SYSTEM_TIME ALL` :

```
USE EcommerceDB;
GO

SELECT ProductID, CurrentPrice, SysStartTime, SysEndTime
FROM ProductPrice
FOR SYSTEM_TIME ALL
WHERE ProductID = 1;
```

T-SQL

Resultado de la auditoría temporal:



The screenshot shows the SQL Server Enterprise Manager (SSMS) interface. On the left is the 'Explorador de objetos' (Object Explorer) showing the database structure of 'EcommerceDB'. The central pane displays a SQL query in a window titled 'SQLQuery4.s...DB (sa (53))*'. The query is as follows:

```
1  USE EcommerceDB;
2  GO
3
4  -- Query price history
5  SELECT ProductID, CurrentPrice, SysStartTime, SysEndTime
6  FROM ProductPrice
7  FOR SYSTEM_TIME ALL
8  WHERE ProductID = 1;
```

Below the query editor, the 'Resultados' (Results) tab is active, showing a single row of data for ProductID 1. The columns are ProductID, CurrentPrice, SysStartTime, and SysEndTime.

ProductID	CurrentPrice	SysStartTime	SysEndTime
1	109.99	2026-09-21 08:30:10.8086616	9999-12-31 23:59:59.9999999

Figura 5. Evidencia en SSMS de la consulta de trazabilidad histórica de precios.

5 Almacenamiento de Metadatos con Tipo Nativo JSON

Uso del tipo nativo `JSON` incorporado en SQL Server 2025, indexación mediante columna calculada y consultas puntuales con `JSON_VALUE`.

T-SQL

```
USE EcommerceDB;
GO

-- Add metadata column to Product (JSON type requires SQL Server 2025)
ALTER TABLE Product ADD Metadata JSON;
GO

-- Add computed column for indexing
ALTER TABLE Product ADD MetadataColor AS JSON_VALUE(Metadata, '$.color');
GO

-- Create index on the computed column
CREATE NONCLUSTERED INDEX IX_Product_Metadata_Color
    ON Product (MetadataColor);
GO

-- Update products with metadata
UPDATE Product SET Metadata = N'{"color":"blue","size":"large","material":"cotton"}'
WHERE ProductID = 1;

UPDATE Product SET Metadata = N'{"color":"red","size":"small","material":"silk"}'
WHERE ProductID = 2;
GO
```

Resultado de la configuración JSON:

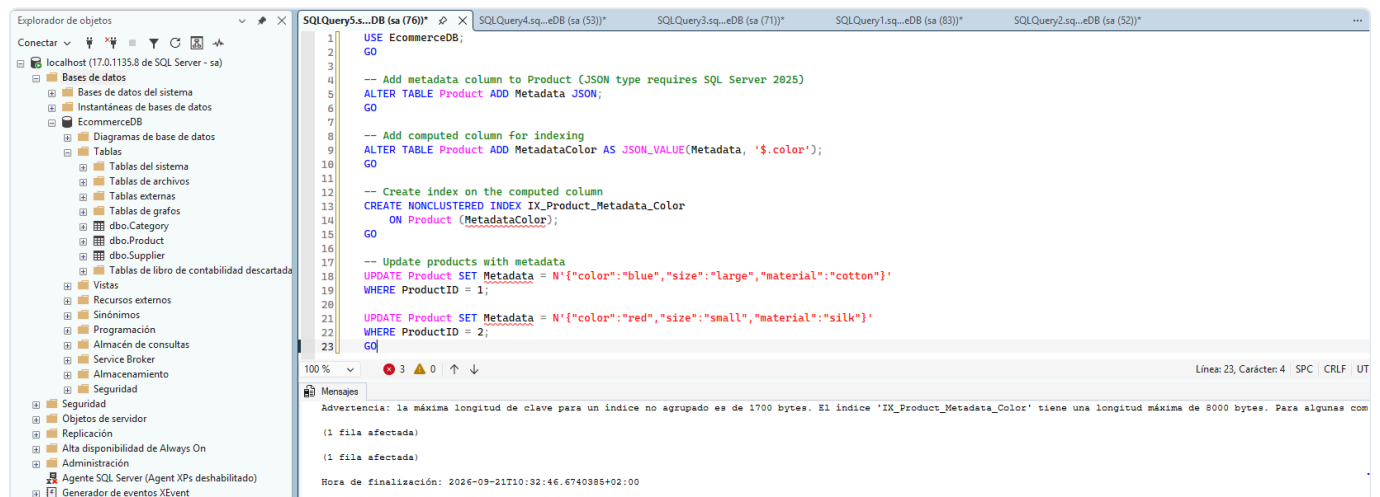


Figura 6. Evidencia en SSMS de la modificación del esquema y de la creación del índice sobre metadatos JSON.

Consulta optimizada extrayendo propiedades mediante rutas JSON:

```
USE EcommerceDB;
GO

SELECT
    ProductID,
    ProductName,
    JSON_VALUE(Metadata, '$.color') AS Color,
    JSON_VALUE(Metadata, '$.size') AS Size,
    JSON_VALUE(Metadata, '$.material') AS Material
FROM Product
WHERE JSON_VALUE(Metadata, '$.color') = 'blue';
```

T-SQL

Resultado de la consulta JSON:

The screenshot displays the SQL Server Enterprise Manager (SSMS) interface. On the left, the 'Explorador de objetos' (Object Explorer) shows the database structure for 'EcommerceDB', including tables like 'dbo.Category', 'dbo.Product', and 'dbo.Supplier'. The main window shows a SQL query in the 'SQLQuery6.s...DB (sa (54))*' tab. The query is identical to the one in the previous block. Below the query editor, the 'Resultados' (Results) pane shows a single row of data for ProductID 1, with columns ProductName, Color, Size, and Material. The data values are 'Wireless Mouse', 'blue', 'large', and 'cotton' respectively.

ProductID	ProductName	Color	Size	Material
1	Wireless Mouse	blue	large	cotton

Figura 7. Evidencia en SSMS de la consulta de atributos JSON mediante JSON_VALUE.

6 Particionamiento de Tablas Masivas (Order)

Segmentación horizontal de la tabla de pedidos mediante una función de partición por rangos de fecha (`RANGE RIGHT`) y un esquema de partición alineado sobre el grupo de archivos principal.

T-SQL

```
USE EcommerceDB;
GO

-- Create partition function for order dates
CREATE PARTITION FUNCTION PF_OrderDate (DATE)
    AS RANGE RIGHT FOR VALUES
        ('2025-01-01', '2025-04-01', '2025-07-01', '2025-10-01');

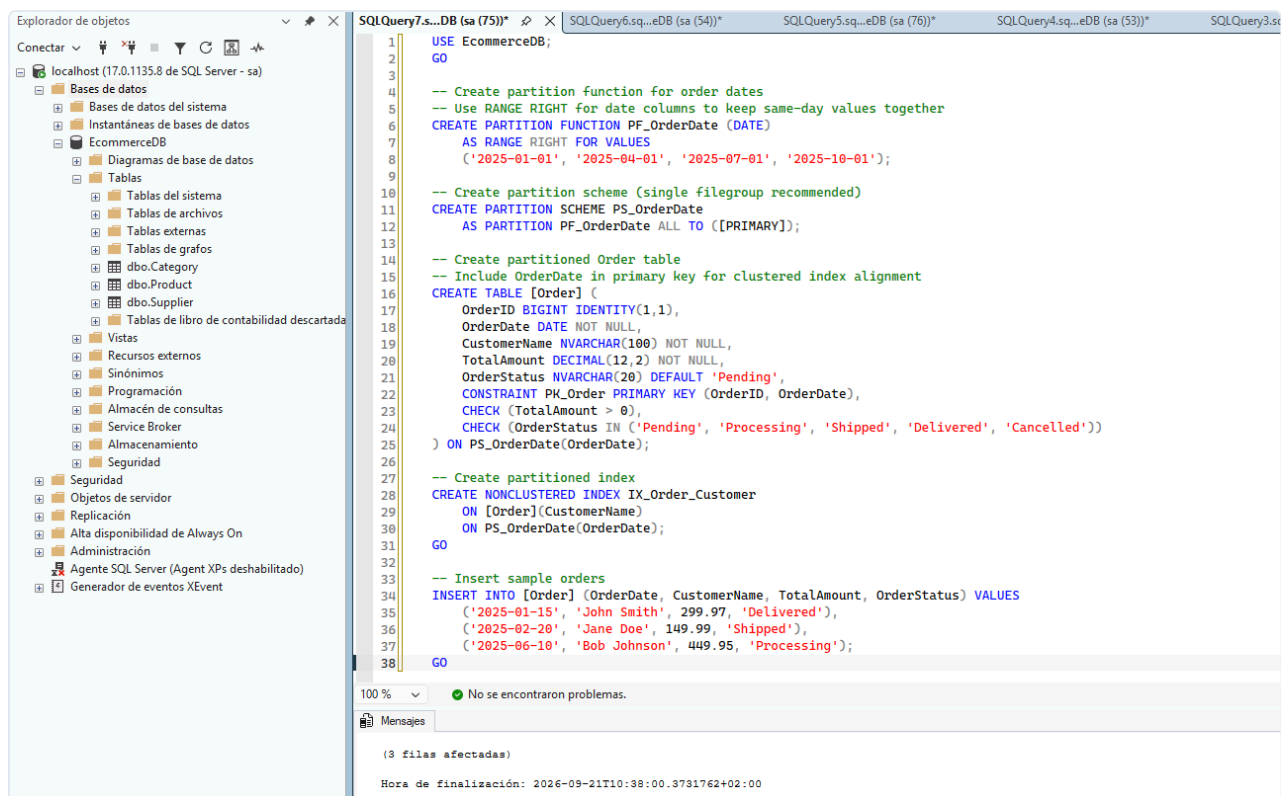
-- Create partition scheme (single filegroup recommended)
CREATE PARTITION SCHEME PS_OrderDate
    AS PARTITION PF_OrderDate ALL TO ([PRIMARY]);

-- Create partitioned Order table
CREATE TABLE [Order] (
    OrderID BIGINT IDENTITY(1,1),
    OrderDate DATE NOT NULL,
    CustomerName NVARCHAR(100) NOT NULL,
    TotalAmount DECIMAL(12,2) NOT NULL,
    OrderStatus NVARCHAR(20) DEFAULT 'Pending',
    CONSTRAINT PK_Order PRIMARY KEY (OrderID, OrderDate),
    CHECK (TotalAmount > 0),
    CHECK (OrderStatus IN ('Pending', 'Processing', 'Shipped', 'Delivered', 'Cancelled'))
) ON PS_OrderDate(OrderDate);

-- Create partitioned index
CREATE NONCLUSTERED INDEX IX_Order_Customer
    ON [Order](CustomerName)
    ON PS_OrderDate(OrderDate);
GO

-- Insert sample orders
INSERT INTO [Order] (OrderDate, CustomerName, TotalAmount, OrderStatus) VALUES
    ('2025-01-15', 'John Smith', 299.97, 'Delivered'),
    ('2025-02-20', 'Jane Doe', 149.99, 'Shipped'),
    ('2025-06-10', 'Bob Johnson', 449.95, 'Processing');
GO
```

Resultado de creación y carga en tabla particionada:



The screenshot displays the SQL Server Enterprise Manager (SSMS) interface. On the left, the 'Explorador de objetos' (Object Explorer) shows the database structure for 'EcommerceDB', including tables like 'dbo.Category', 'dbo.Product', and 'dbo.Supplier'. The right pane shows the SQL script for creating a partitioned table 'Order' and inserting sample data.

```
1 USE EcommerceDB;
2 GO
3
4 -- Create partition function for order dates
5 -- Use RANGE RIGHT for date columns to keep same-day values together
6 CREATE PARTITION FUNCTION PF_OrderDate (DATE)
7 AS RANGE RIGHT FOR VALUES
8 ('2025-01-01', '2025-04-01', '2025-07-01', '2025-10-01');
9
10 -- Create partition scheme (single filegroup recommended)
11 CREATE PARTITION SCHEME PS_OrderDate
12 AS PARTITION PF_OrderDate ALL TO ([PRIMARY]);
13
14 -- Create partitioned Order table
15 -- Include OrderDate in primary key for clustered index alignment
16 CREATE TABLE [Order] (
17     OrderID BIGINT IDENTITY(1,1),
18     OrderDate DATE NOT NULL,
19     CustomerName NVARCHAR(100) NOT NULL,
20     TotalAmount DECIMAL(12,2) NOT NULL,
21     OrderStatus NVARCHAR(20) DEFAULT 'Pending',
22     CONSTRAINT PK_Order PRIMARY KEY (OrderID, OrderDate),
23     CHECK (TotalAmount > 0),
24     CHECK (OrderStatus IN ('Pending', 'Processing', 'Shipped', 'Delivered', 'Cancelled'))
25 ) ON PS_OrderDate(OrderDate);
26
27 -- Create partitioned index
28 CREATE NONCLUSTERED INDEX IX_Order_Customer
29 ON [Order](CustomerName)
30 ON PS_OrderDate(OrderDate);
31 GO
32
33 -- Insert sample orders
34 INSERT INTO [Order] (OrderDate, CustomerName, TotalAmount, OrderStatus) VALUES
35 ('2025-01-15', 'John Smith', 299.97, 'Delivered'),
36 ('2025-02-20', 'Jane Doe', 149.99, 'Shipped'),
37 ('2025-06-10', 'Bob Johnson', 449.95, 'Processing');
38 GO
```

100 %  No se encontraron problemas.

Mensajes

(3 filas afectadas)

Hora de finalización: 2026-09-21T10:38:00.3731762+02:00

Figura 8. Evidencia en SSMS de la creación y carga de la tabla de pedidos particionada.

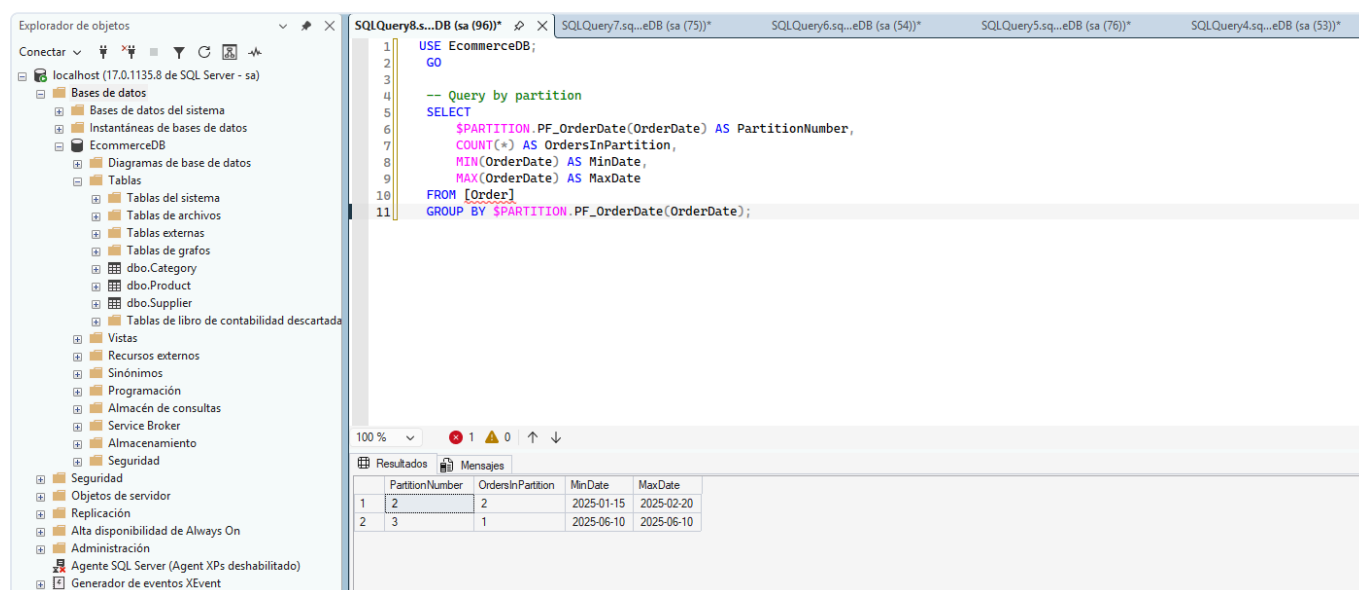
Inspección de la distribución de filas por número de partición con `$PARTITION` :

```
USE EcommerceDB;
GO

SELECT
    $PARTITION.PF_OrderDate(OrderDate) AS PartitionNumber,
    COUNT(*) AS OrdersInPartition,
    MIN(OrderDate) AS MinDate,
    MAX(OrderDate) AS MaxDate
FROM [Order]
GROUP BY $PARTITION.PF_OrderDate(OrderDate);
```

T-SQL

Resultado de distribución de particiones:



Explorador de objetos

Conectar

localhost (17.0.1135.8 de SQL Server - sa)

Bases de datos

Bases de datos del sistema

Instantáneas de bases de datos

EcommerceDB

Diagramas de base de datos

Tablas

Tablas del sistema

Tablas de archivos

Tablas externas

Tablas de grafos

dbo.Category

dbo.Product

dbo.Supplier

Tablas de libro de contabilidad descartada

Vistas

Recursos externos

Sinónimos

Programación

Almacén de consultas

Service Broker

Almacenamiento

Seguridad

Seguridad

Objetos de servidor

Replicación

Alta disponibilidad de Always On

Administración

Agente SQL Server (Agent XPs deshabilitado)

Generador de eventos XEvent

SQLQuery8.s...DB (sa (96))

SQLQuery7.sq...eDB (sa (75))

SQLQuery6.sq...eDB (sa (54))

SQLQuery5.sq...eDB (sa (76))

SQLQuery4.sq...eDB (sa (53))

```
1 USE EcommerceDB;
2 GO
3
4 -- Query by partition
5 SELECT
6     $PARTITION.PF_OrderDate(OrderDate) AS PartitionNumber,
7     COUNT(*) AS OrdersInPartition,
8     MIN(OrderDate) AS MinDate,
9     MAX(OrderDate) AS MaxDate
10 FROM [Order]
11 GROUP BY $PARTITION.PF_OrderDate(OrderDate);
```

100 %

1 0

Resultados Mensajes

	PartitionNumber	OrdersInPartition	MinDate	MaxDate
1	2	2	2025-01-15	2025-02-20
2	3	1	2025-06-10	2025-06-10

Figura 9. Evidencia en SSMS de la distribución de filas por número de partición.

7 Generación de Claves Independientes mediante SEQUENCE

Creación de un generador numérico desacoplado (`OrderLineSequence`) para alimentar la clave primaria de los detalles de pedido (`OrderDetail`) mediante `NEXT VALUE FOR` .

T-SQL

```
USE EcommerceDB;
GO

-- Create SEQUENCE for order line items
CREATE SEQUENCE OrderLineSequence
    START WITH 1
    INCREMENT BY 1;

-- Create OrderDetail table
CREATE TABLE OrderDetail (
    OrderLineID INT PRIMARY KEY,
    OrderID BIGINT NOT NULL,
    OrderDate DATE NOT NULL,
    ProductID INT NOT NULL,
    Quantity INT NOT NULL,
    UnitPrice DECIMAL(10,2) NOT NULL,
    LineTotal AS (Quantity * UnitPrice),
    CHECK (Quantity > 0),
    CHECK (UnitPrice > 0),
    FOREIGN KEY (OrderID, OrderDate) REFERENCES [Order](OrderID, OrderDate),
    FOREIGN KEY (ProductID) REFERENCES Product(ProductID)
);
GO

-- Insert order details using SEQUENCE
INSERT INTO OrderDetail (OrderLineID, OrderID, OrderDate, ProductID, Quantity, UnitPrice)
VALUES
    (NEXT VALUE FOR OrderLineSequence, 1, '2025-01-15', 1, 2, 99.99),
    (NEXT VALUE FOR OrderLineSequence, 1, '2025-01-15', 2, 1, 149.99),
    (NEXT VALUE FOR OrderLineSequence, 2, '2025-02-20', 1, 3, 99.99);
GO
```

Resultado de la inserción secuencial:

Explorador de objetos

Conectar

localhost (17.0.1135.8 de SQL Server - sa)

Bases de datos

Instantáneas de bases de datos

EcommerceDB

Diagramas de base de datos

Tablas

Tablas del sistema

Tablas de archivos

Tablas externas

Tablas de grafos

dbo.Category

dbo.Product

dbo.Supplier

Tablas de libro de contabilidad descartada

Vistas

Recursos externos

Sinónimos

Programación

Almacén de consultas

Service Broker

Almacenamiento

Seguridad

Seguridad

Objetos de servidor

Replicación

Alta disponibilidad de Always On

Administración

Agente SQL Server (Agent XPs deshabilitado)

Generador de eventos XEvent

SQLQuery9.s...DB (sa (56))*

SQLQuery8.sq...eDB (sa (96))*

SQLQuery7.sq...eDB (sa (75))*

SQLQuery6.sq...eDB (sa (54))*

```
9 -- Create OrderDetail table
10 CREATE TABLE OrderDetail (
11     OrderLineID INT PRIMARY KEY,
12     OrderID BIGINT NOT NULL,
13     OrderDate DATE NOT NULL,
14     ProductID INT NOT NULL,
15     Quantity INT NOT NULL,
16     UnitPrice DECIMAL(10,2) NOT NULL,
17     LineTotal AS (Quantity * UnitPrice),
18     CHECK (Quantity > 0),
19     CHECK (UnitPrice > 0),
20     FOREIGN KEY (OrderID, OrderDate) REFERENCES [Order](OrderID, OrderDate),
21     FOREIGN KEY (ProductID) REFERENCES Product(ProductID)
22 );
23 GO
24
25 -- Insert order details using SEQUENCE
26 INSERT INTO OrderDetail (OrderLineID, OrderID, OrderDate, ProductID, Quantity, UnitPrice)
27 VALUES
28     (NEXT VALUE FOR OrderLineSequence, 1, '2025-01-15', 1, 2, 99.99),
29     (NEXT VALUE FOR OrderLineSequence, 1, '2025-01-15', 2, 1, 149.99),
30     (NEXT VALUE FOR OrderLineSequence, 2, '2025-02-20', 1, 3, 99.99);
31 GO
```

100%

1 0

Mensajes

(3 filas afectadas)

Hora de finalización: 2026-09-21T10:41:13.0885660+02:00

Figura 10. Evidencia en SSMS de la creación de la secuencia y de la inserción de detalles de pedido.

Validación de registros e identificación de correlativos generados:

```
USE EcommerceDB;  
GO  
  
SELECT * FROM OrderDetail;
```

T-SQL

Resultado del contenido de OrderDetail:

The screenshot displays the SQL Server Enterprise Manager (SSMS) interface. On the left, the 'Explorador de objetos' (Object Explorer) shows the database structure for 'localhost (17.0.1135.8 de SQL Server - sa)'. The 'Bases de datos' (Databases) folder is expanded, showing 'EcommerceDB'. Under 'EcommerceDB', the 'Tablas' (Tables) folder is expanded, listing tables such as 'dbo.Category', 'dbo.Product', 'dbo.Supplier', and 'dbo.OrderDetail'. The 'dbo.OrderDetail' table is selected. On the right, the 'SQLQuery10....DB (sa (80))' window shows the following T-SQL query:

```
1 USE EcommerceDB;  
2 GO  
3  
4 SELECT * FROM OrderDetail;
```

Below the query editor, the 'Resultados' (Results) tab is active, displaying a table with 8 columns: 'OrderLineID', 'OrderID', 'OrderDate', 'ProductID', 'Quantity', 'UnitPrice', and 'Line Total'. The table contains 3 rows of data:

	OrderLineID	OrderID	OrderDate	ProductID	Quantity	UnitPrice	Line Total
1	1	1	2025-01-15	1	2	99.99	199.98
2	2	1	2025-01-15	2	1	149.99	149.99
3	3	2	2025-02-20	1	3	99.99	299.97

Figura 11. Evidencia en SSMS del contenido de la tabla OrderDetail con los correlativos generados.

8 Verificación de Integridad y Validación de Objetos

Validación de la restricción CHECK

Comprobación de que el motor cancela intentos de inserción con precios base negativos (-50), arrojando el error Mens. 547 de infracción de restricción de comprobación.

```
USE EcommerceDB;
GO

-- This should fail: negative price
INSERT INTO Product (ProductName, CategoryID, SupplierID, BasePrice, StockQuantity)
VALUES ('Invalid', 1, 1, -50, 10);
```

T-SQL

Resultado del error controlado:

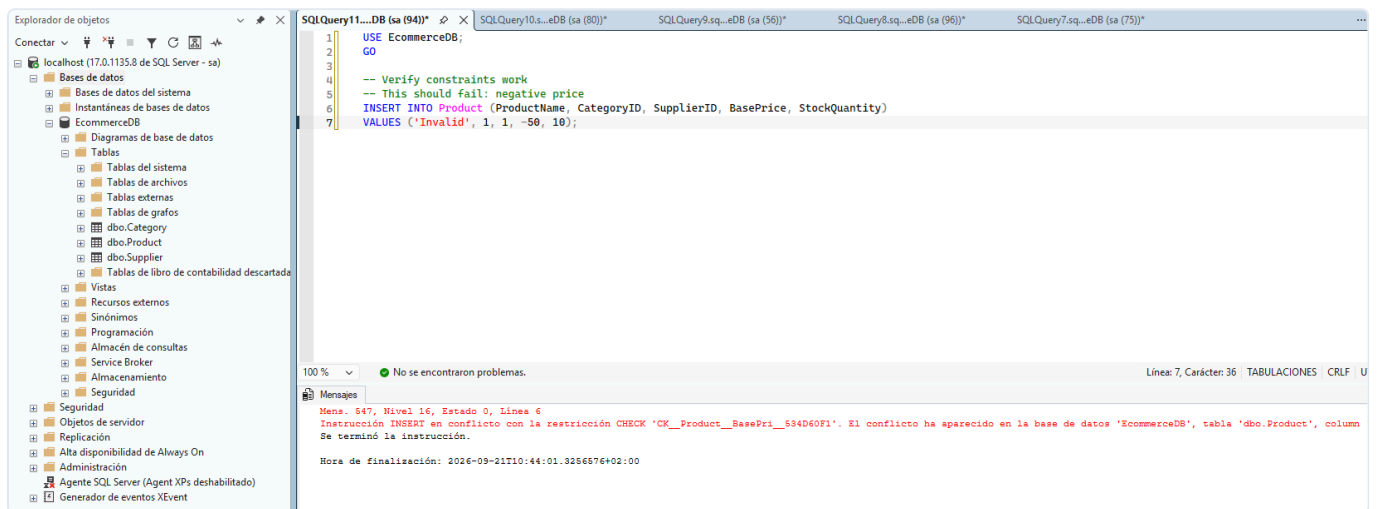


Figura 12. Evidencia en SSMS del error Mens. 547 por infracción de la restricción CHECK.

Comprobación integral de objetos avanzados

Ejecución del lote de validación final para JSON, particiones activas y tablas temporales.

T-SQL

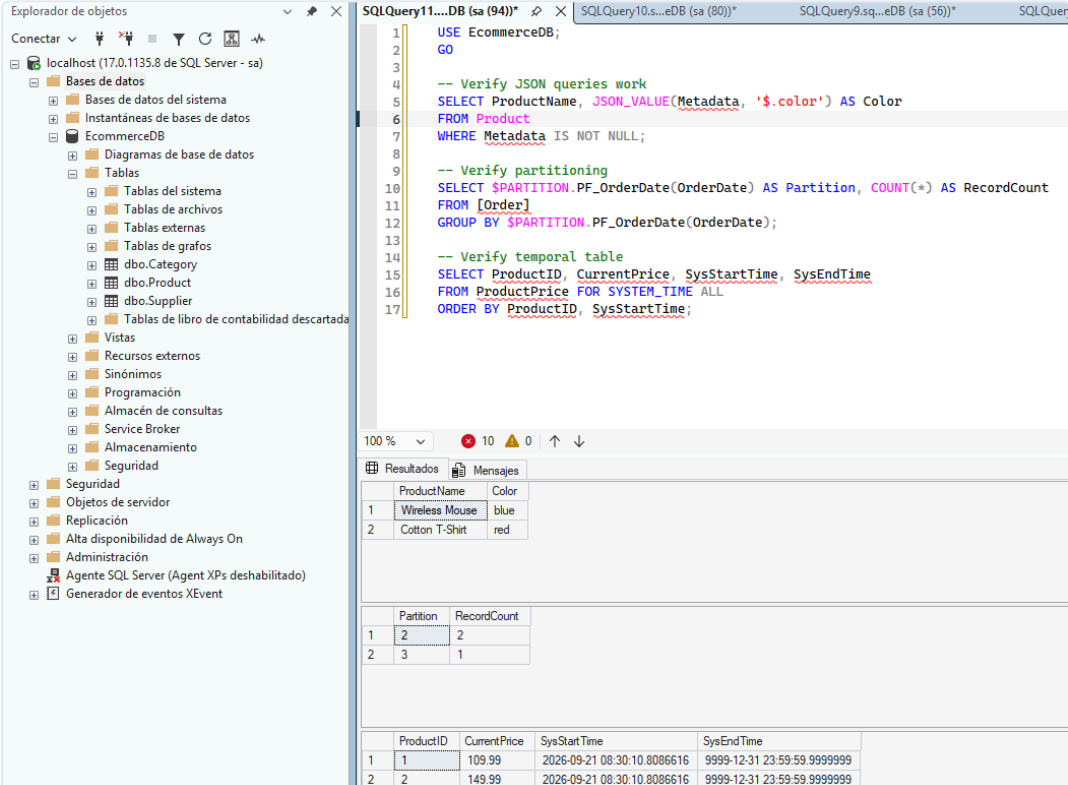
```
USE EcommerceDB;
GO

-- Verify JSON queries work
SELECT ProductName, JSON_VALUE(Metadata, '$.color') AS Color
FROM Product
WHERE Metadata IS NOT NULL;

-- Verify partitioning
SELECT $PARTITION.PF_OrderDate(OrderDate) AS Partition, COUNT(*) AS RecordCount
FROM [Order]
GROUP BY $PARTITION.PF_OrderDate(OrderDate);

-- Verify temporal table
SELECT ProductID, CurrentPrice, SysStartTime, SysEndTime
FROM ProductPrice FOR SYSTEM_TIME ALL
ORDER BY ProductID, SysStartTime;
```

Resultado de la verificación global:



The screenshot shows the SQL Server Enterprise Manager (SSMS) interface. On the left, the 'Explorador de objetos' (Object Explorer) pane displays the database structure for 'EcommerceDB'. The main pane shows a T-SQL script being executed. The script includes comments and SQL statements to verify JSON queries, partitioning, and temporal tables. The results of the execution are displayed in three tables:

ProductID	CurrentPrice	SysStartTime	SysEndTime
1	109.99	2026-09-21 08:30:10.8086616	9999-12-31 23:59:59.9999999
2	149.99	2026-09-21 08:30:10.8086616	9999-12-31 23:59:59.9999999

Figura 13. Evidencia en SSMS de la validación global de JSON, particiones y tablas temporales.